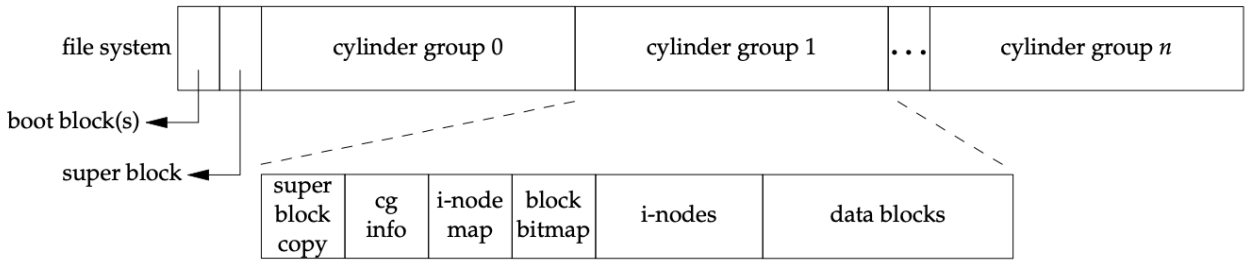
File System Emulation

For this assignment you will work with a partially implemented file system to correct some of its functionality and complement it with more. This will help you to get experience with the concepts of a file system, and how various low-level functions/commands are written. In addition, you will get experience working with a codebase that's not yours.

Task 1: Code Exploration

Your first task is to explore the filesystem and get familiar with its structure and current functionality. The given file system has a structure similar with the one in the book and in the notes. When you compile and run the code, you should see a directory named *"filesystem"* appear inside your working directory. Similarly with the image below, inside the directory you will see two files *"bootsector"* and *"superblock",* and a directory named *"cylinderBlock_0"*.



The *"bootsector"* file is currently empty. The *"superblock"* file holds information for the file system, such as,

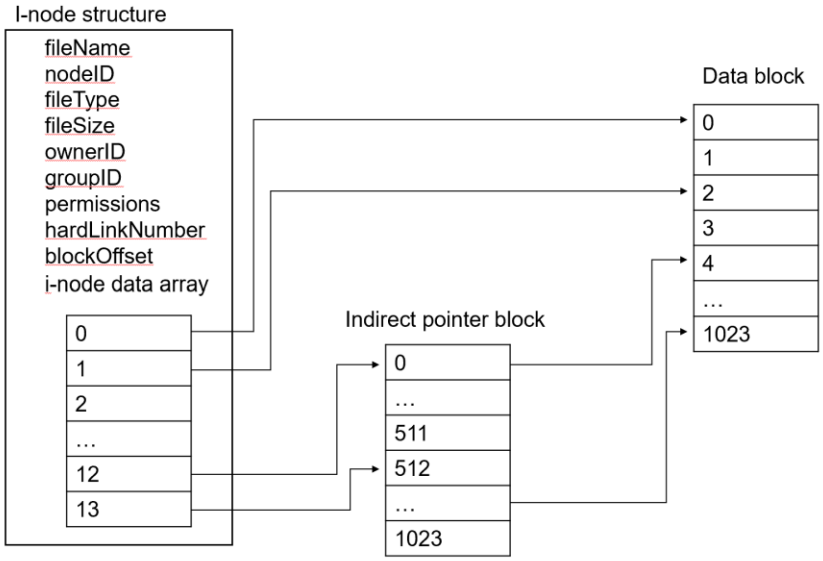
DISK_SIZE → The total size of the disk
CYL_BLOCK_NUM → The cylinder number
BLOCK_SIZE → The size of each data block
BLOCK_NUM → The number of the data blocks
INODE_NUM → The inode number
INODE_PTR_ARRAY_SIZE → The size of the inode pointer array
INODE_INDIRECT_PTRS → The number of the indirect pointers pointed by the doubly pointers
int inodeidx → The used inode count

The "cylinderBlock_0" folder contains the folders *"datablocks"* and *"indirectptsblock"*, and the files *"blockbitmap"*, *"cylindergroupinfo"*, *"inodearray"*, *"inodebitmap"*, and *"superblockcopy"*.

These files should be known to you based on what we learned in the class. The folders are there to keep the data and indirect pointed blocks grouped together.

When you run the program, you will see information about its functionality. Not every function is implemented. You can make directories with the mkDir command and use the exit command to exit the program.

The structure of the i-nodes in the file system is shown below.



Task 2: Improve mkDir function

The current implementation of the file system does not create new hard links from the parent directory to the new directory inside the parent directory's data blocks. It creates a hard link directly in the parent directory's i-node data array, which limits the i-node's capacity. Also, it doesn't update the hard link number of the parent directory when a subdirectory is created, and it doesn't update the i-node bitmap. Your task is to modify the code and correct the implementation of the function.

To create hard links in the data blocks, you will need to check if the parent directory already uses a data block. If it does, then you need to add the new i-node inside that data block. If it doesn't then you need to scan the block bitmap and find the next available data block. Mark that block as used, and use its index to open the corresponding data block file and add the new i-node in it. Then you need to create a connection from the parent directory's data array to that data block.

To update the parent directory's hard link count, you need to find the parent i-node inside the i-node array. Then on that line you need to find the hard link number and increase it by one. I would suggest reading the current implementation to see how it adds the hard link because there are some useful details that will help you avoid mistakes in your task, such as data misalignment from added characters in the file.

To update the i-node bitmap, you need to scan the i-node array file and track the index of the new i-node. Then, using that index change the value on the corresponding line in the i-node file to indicate that the i-node is in use.

Task 3: Imlement the mkFile function

The current implementation of the file system does not include implementation for the mkFile function. Your task is to implement the function, so the users can add files to the file system. The mkFile is very similar to the mkDir function, so you should be able to re-use some of its code.

Task 4: Imlement the printINodeInfo function

The current implementation of the file system does not include implementation for the printINodeInfo function. Your task is to implement the function, so the users can see i-node details on the screen.

Task 5: Attempt Extra Credit

The current implementation of the file system does not include implementation for the writeFile, readFile and chDir functions. These are possible extra credits that are worth 1% of the total grade for the class if you complete them!

For all the tasks, your implementations should include all the safety precautions, and it should not have any memory leaks!

Grading (100 pts) :

- Task 2: 40 points
- Task 3: 20 points
- Task 4: 20 points
- Style & Documentation – 20 pts

What to submit:

- The filesystem.c file and your compile script.

Extra Credit:

Once you're done and have submitted everything, there's one last thing that takes 5-10 minutes! As Silas announced in class, he is running research on how students use AI. Please use this link below to either complete the research study or do the alternative reflection activity. Participation in the research is voluntary and confidential to the course instructors, but you must either complete the survey or the reflection activity and explicitly decline participation to receive credit.

https://stevens.qualtrics.com/jfe/form/SV_42w3fWc8opmT0l8?Course=CSC2050-07&Assignment=4